CSC 402 — Mobile Application Programming

Part A — Kotlin Fundamentals

Task 1 — Variables and string templates

Variables and string templates: enrollment changes from 28 to 30; university name length is 38.

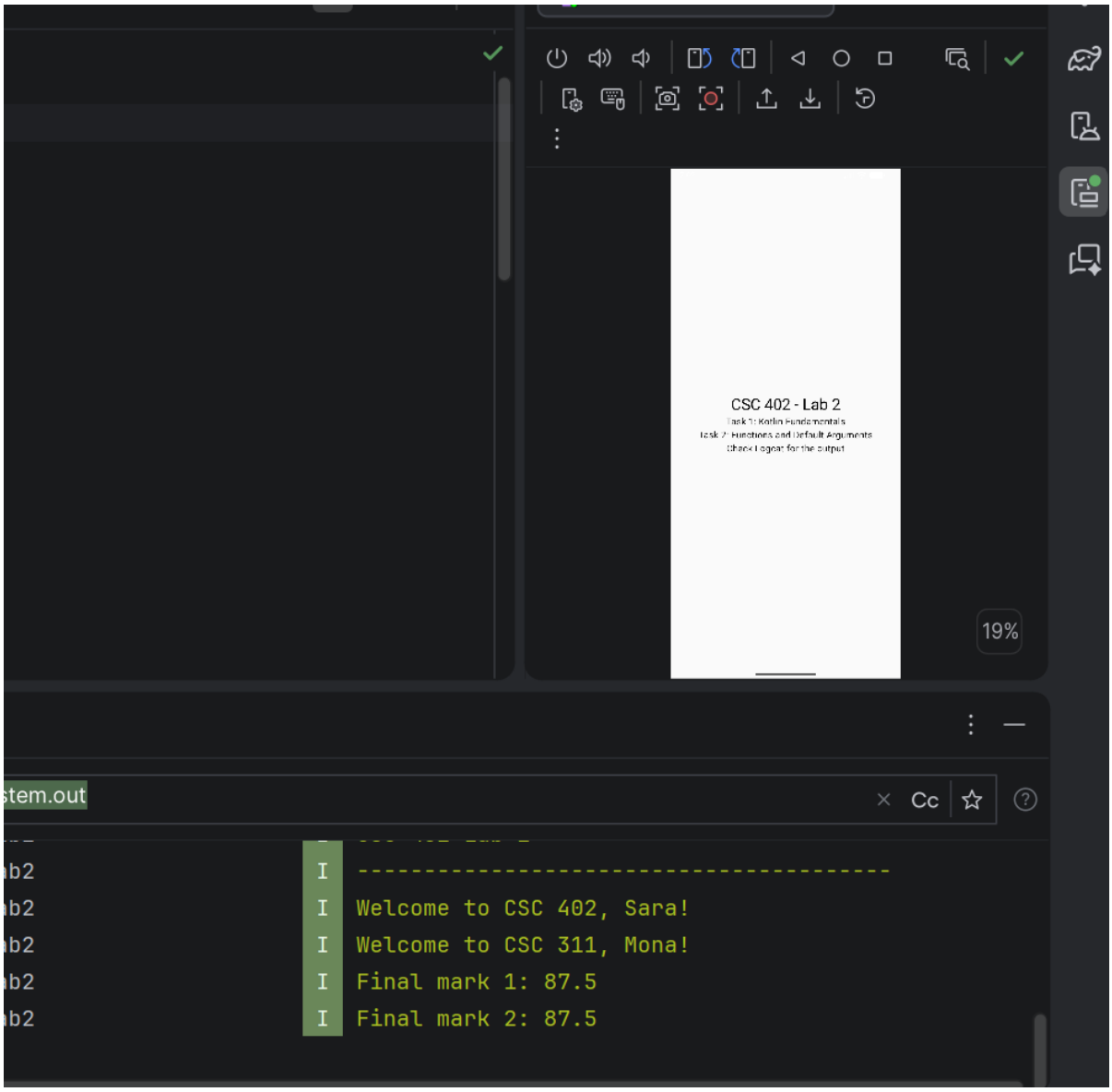


Figure A1.1 — Task 1 Logcat evidence

Task 2 — Functions and default arguments

Functions and default arguments: greetings and weighted marks; both final marks equal 87.5. No separate console screenshot is required.

Task 3 — If and when

If and when: letter grades and pass/fail. The invalid mark 130 currently produces Invalid (Pass), showing a validation inconsistency.

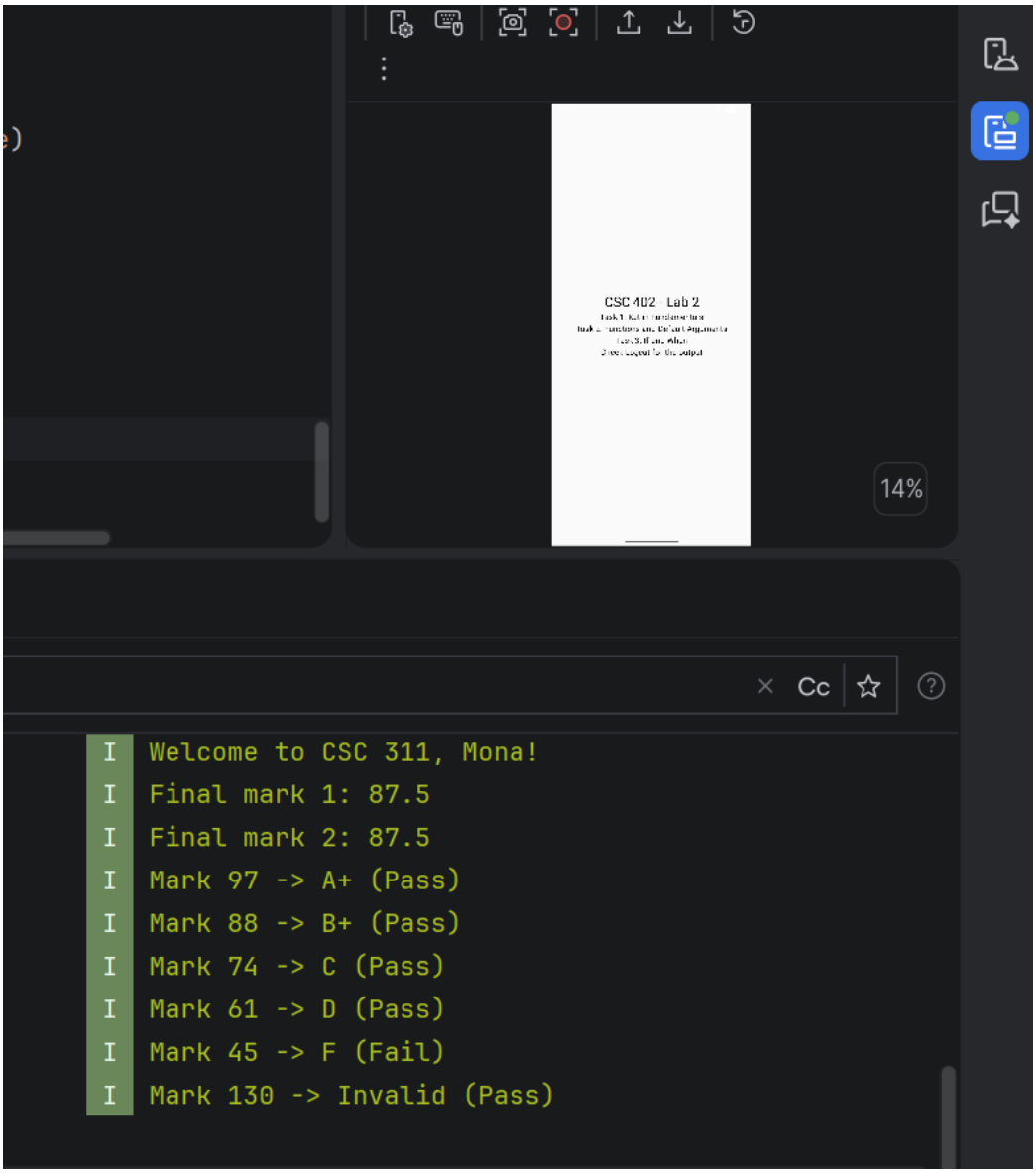


Figure A3.1 — Task 3 Logcat evidence

Task 4 — Null safety

Null safety: safe calls, Elvis operator, and nullable team descriptions.

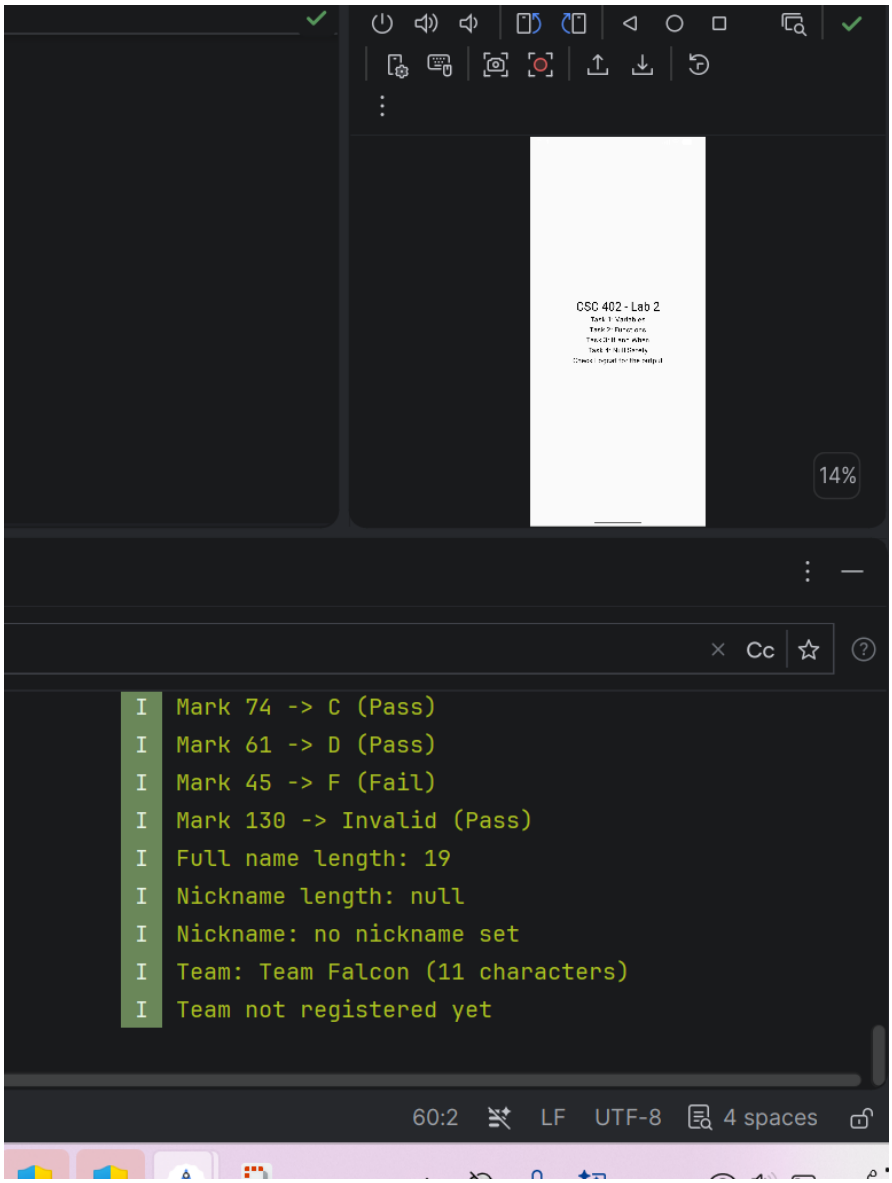


Figure A4.1 — Task 4 Logcat evidence

Task 5 — Data classes and collections

Data classes and collections: course filtering, code mapping, 15 total credits, searching, and immutable copy.

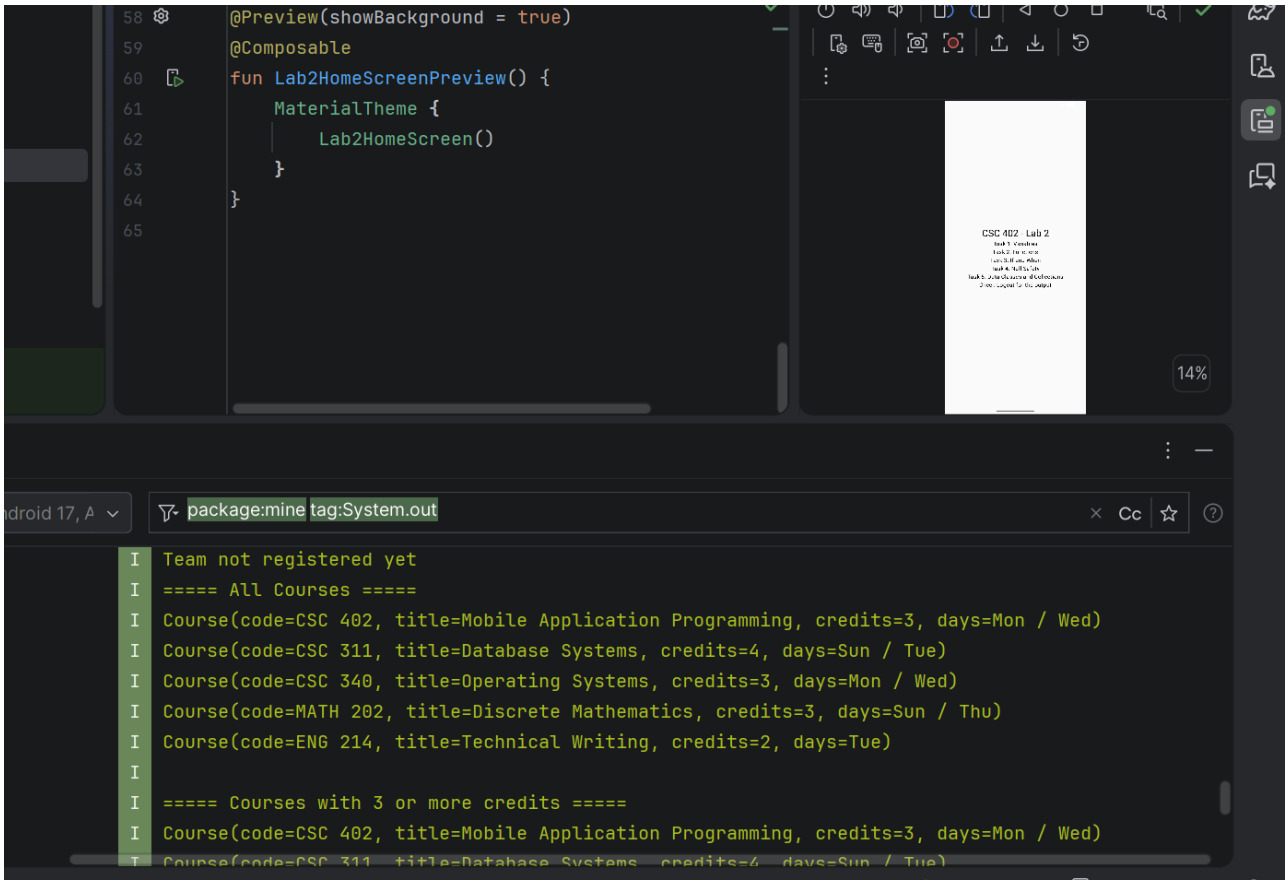


Figure A5.1 — Task 5 Logcat evidence

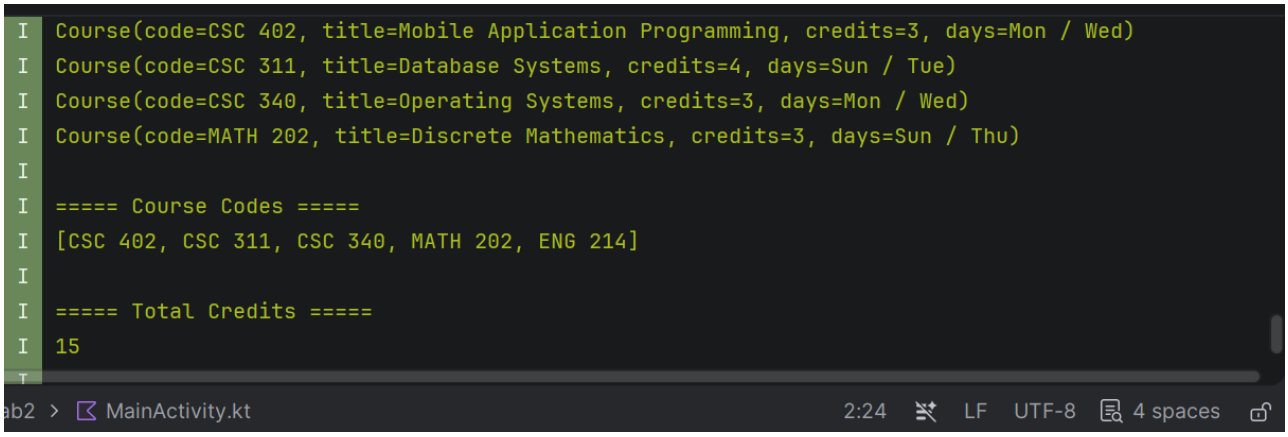


Figure A5.2 — Task 5 Logcat evidence

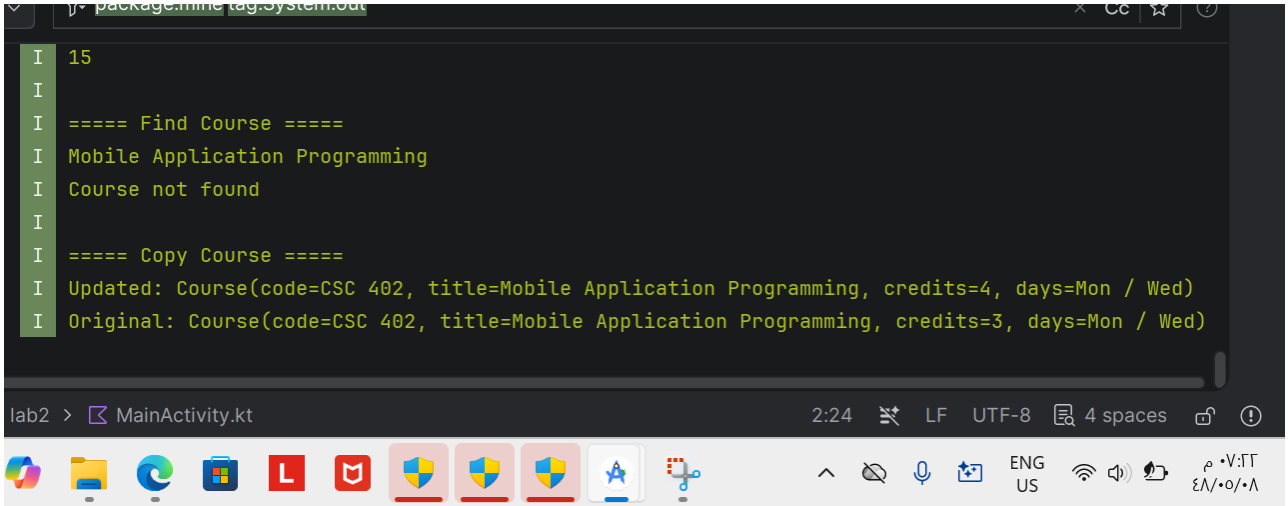


Figure A5.3 — Task 5 Logcat evidence

Part B — Jetpack Compose

Task 6 — Welcome Screen

A Compose welcome screen with header, circular monogram, greeting, and green label.

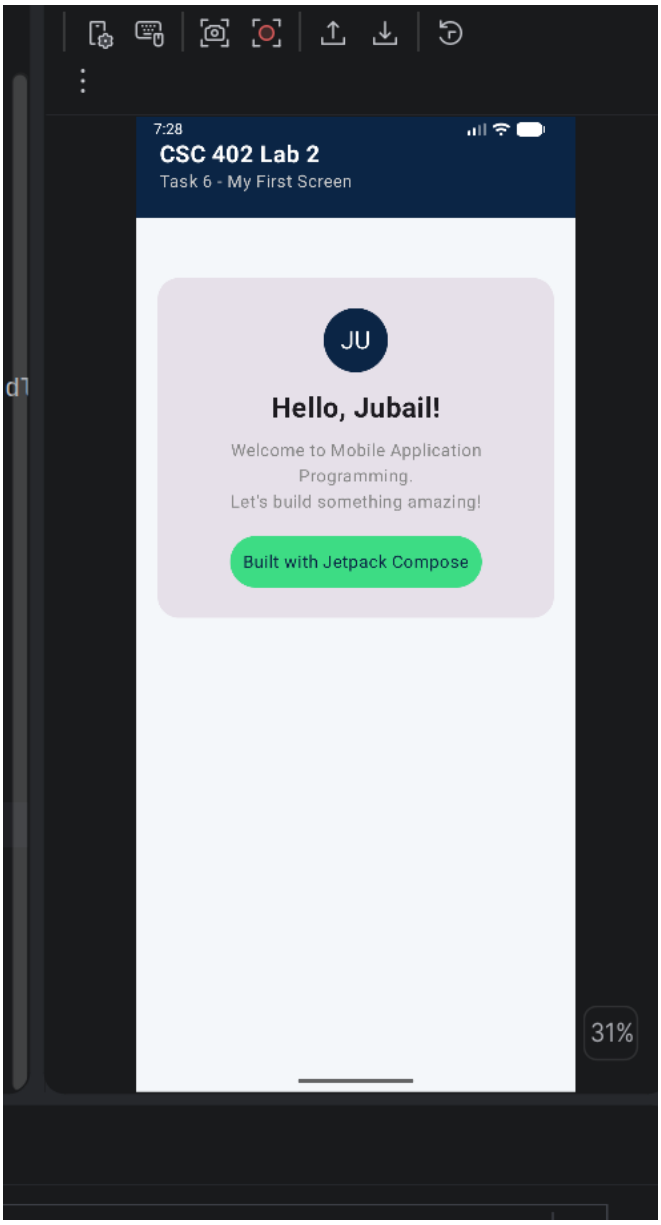


Figure B6.1 — Emulator screenshot for Task 6

Task 7 — Student ID Card

A Row/Column-based student card and a reusable statistics composable. Placeholder student details must be replaced if required.

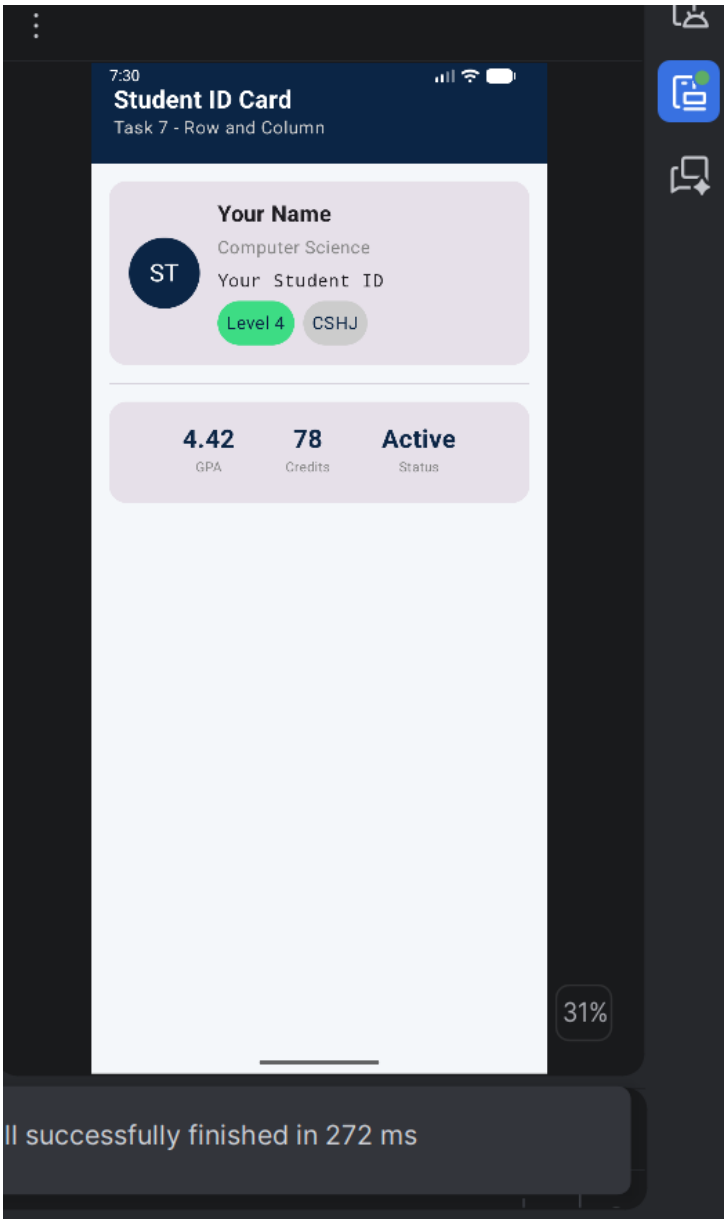


Figure B7.1 — Emulator screenshot for Task 7

Task 8 — Attendance Counter

A reactive attendance count with increment, decrement, and reset controls. The zero state disables decrement and reset.

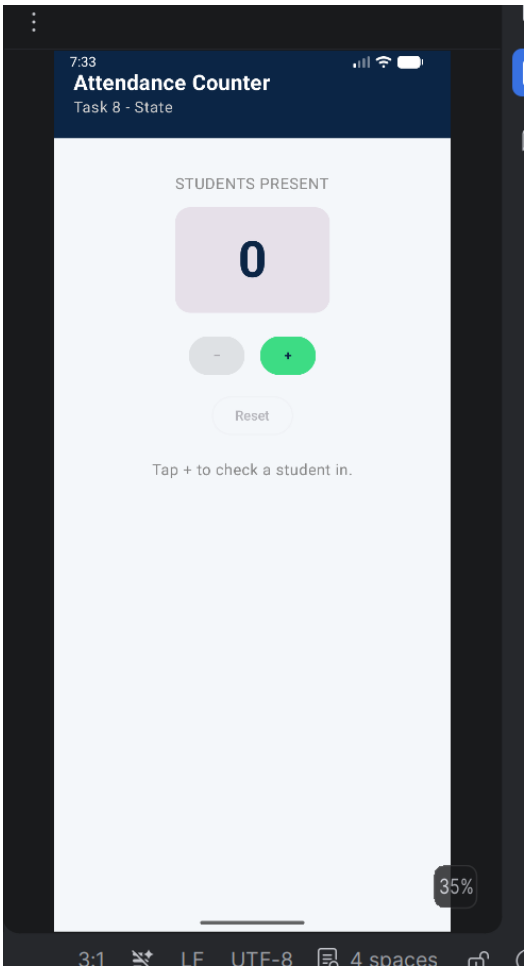


Figure B8.1 — Emulator screenshot for Task 8

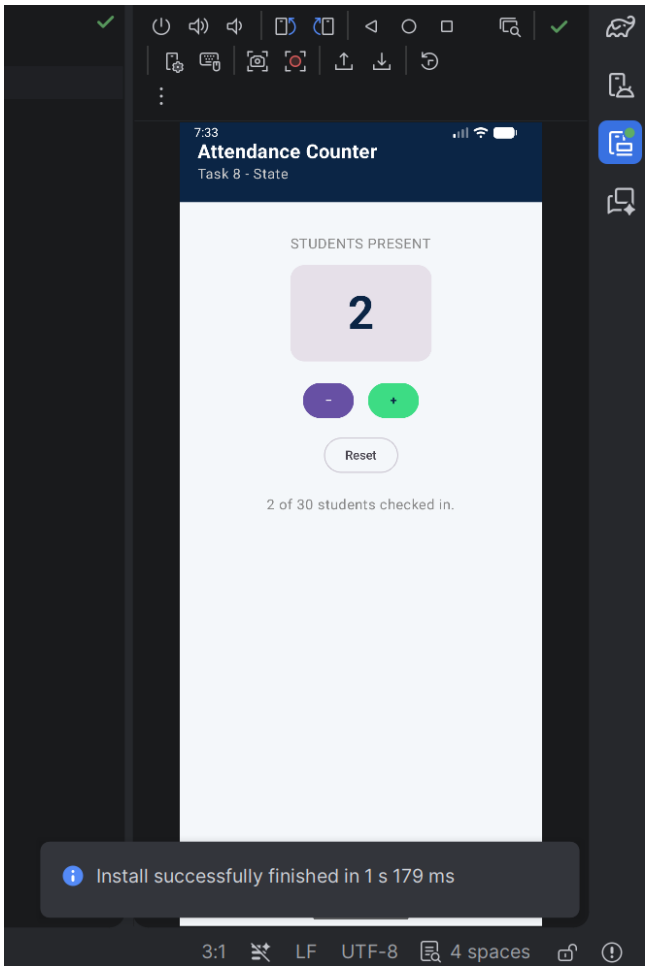


Figure B8.2 — Emulator screenshot for Task 8

Task 9 — Greeting Card

A name input, disabled empty-input button, and a reactive submitted greeting.

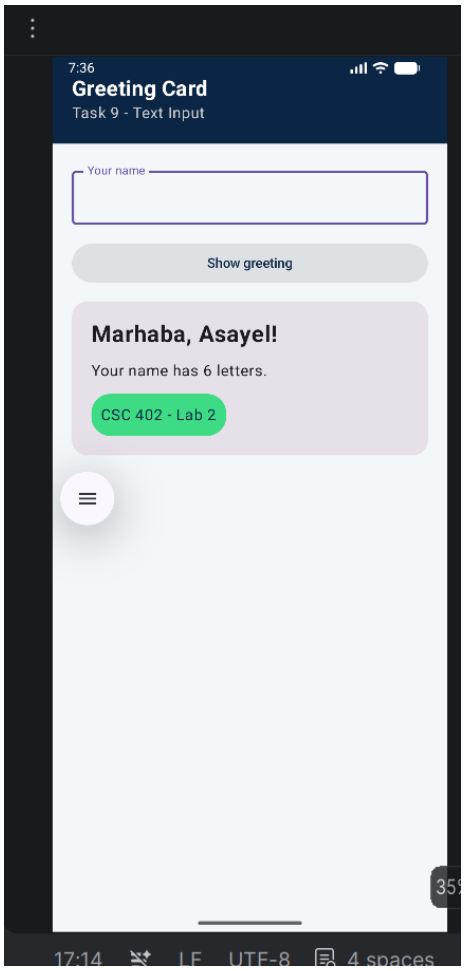


Figure B9.1 — Emulator screenshot for Task 9

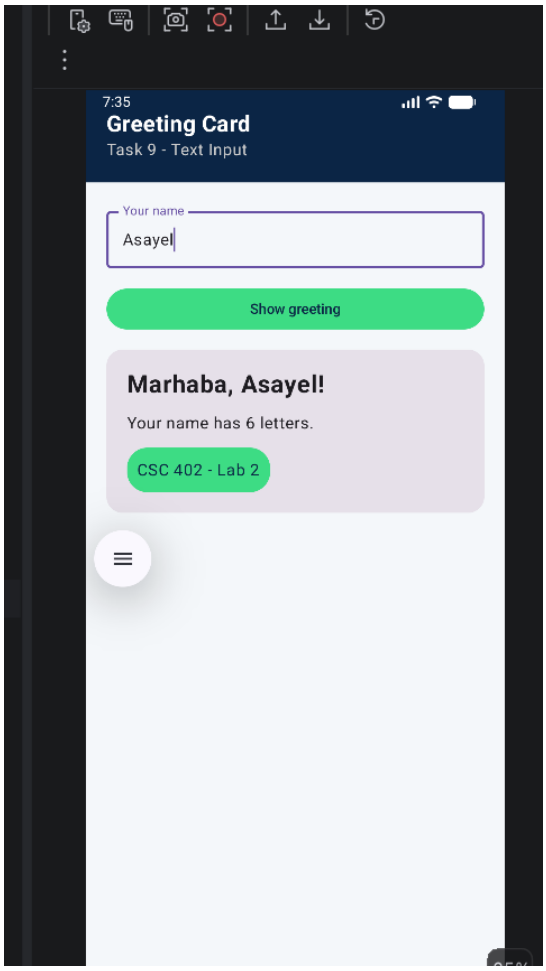


Figure B9.2 — Emulator screenshot for Task 9

Task 10 — Course List

Seven course cards rendered in a LazyColumn with a reusable CourseRow.

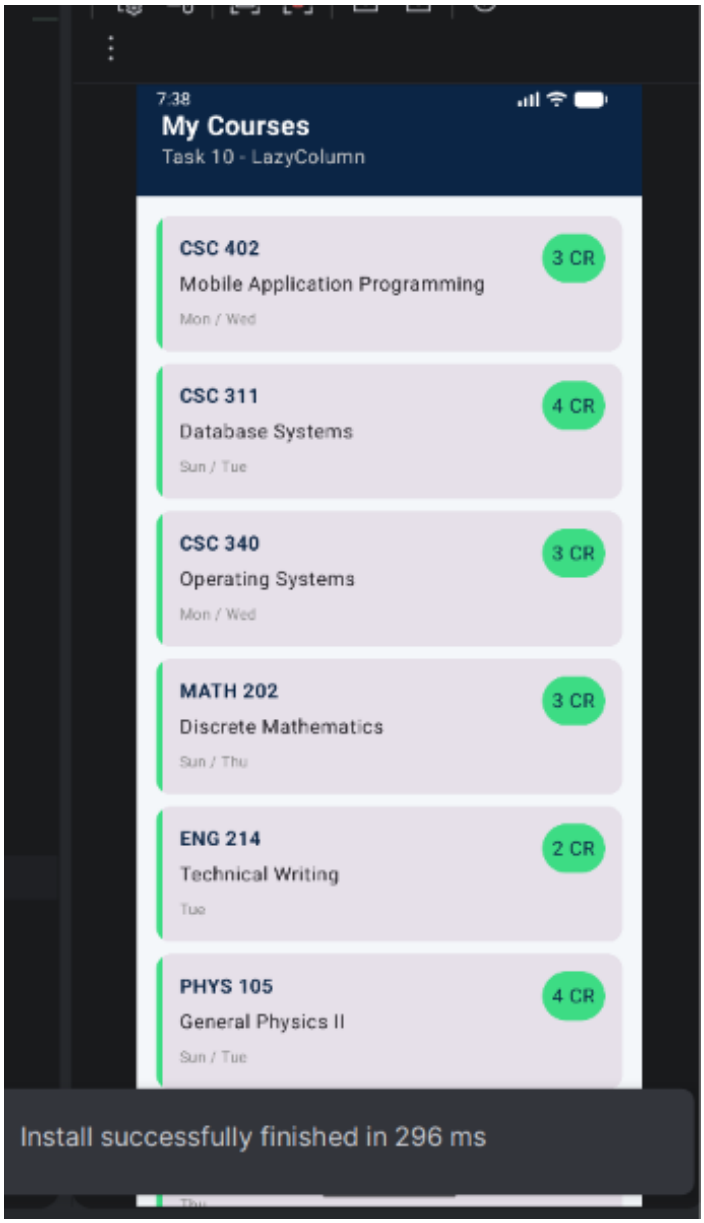


Figure B10.1 — Emulator screenshot for Task 10

Written Questions — Q1 to Q8

Q1. I used var for studentsEnrolled because enrollment changes. The other variables are val because they are intended to stay constant. Reassigning a val produces the compiler error “Val cannot be reassigned”; this exact error has not been independently captured in the submitted screenshots.

Q2. Named arguments make calls easier to understand by explicitly connecting each value to its parameter. They also prevent accidental swapping of same-type parameters, even when arguments are supplied in a different order.

Q3. The invalid mark 130 is labelled Invalid by letterGrade but Pass by status in the current implementation. I would validate that the mark is within 0..100 in status first and return Invalid when it is outside that range.

Q4. Using nickname!!.length when nickname is null throws a NullPointerException on that expression. The screenshot shows the safe version; the crash line was left commented out. The !! operator is discouraged because it can crash the app.

Q5. No, the original CSC 402 course in the list still has 3 credits. copy(credits = 4) creates a separate Course instance, showing that the original data is not changed.

Q6. A plain var is not observable Compose state, so changing it does not trigger recomposition. mutableStateOf makes changes observable; remember keeps the state object across recompositions.

Q7. LazyColumn composes visible and nearby rows as needed rather than composing all rows immediately. This is more memory-efficient and scales better for a list of 200 courses than a vertically scrolling Column.

Q8. TaskRow should receive isDone through its task parameter and send changes through onToggle. If each row privately owned the completion state, the parent summary strip could not reliably calculate Done and Open counts from the shared list.